Project Phase 3

Samia Zaman

University of the Cumberlands

MSCS-532-B01: Algorithms and Data Structures

Prof. Michael Solomon

April 5, 2026

Optimization Techniques

Phase 2 left three bottlenecks unresolved:

1) First, the nearest-unit lookup ran one full Dijkstra per available unit, costing O(n * (V+E) log V) per dispatch where n is fleet size. At n=200 on the benchmark grid this translated to roughly 29ms per dispatch call.
2) Second, the road network used fixed edge weights - there was no way to model traffic or road closures.
3) Finally, there was no systematic benchmark comparing the implementation against a baseline at scale.

Phase 3 addresses all three.

The main optimization is a 2D k-d tree spatial index (KDTree class) over the positions of available units (Bentley, 1975). Each road network location is assigned (x, y) coordinates. When a call comes in, the system queries the k-d tree for the k Euclidean-nearest available units (default k=3), then runs Dijkstra only for those k candidates. The tree is rebuilt from scratch after each dispatch. Total cost per dispatch: O(n log n) for the rebuild, O(log n) for the tree query, and O(k * (V+E) log V) for the k Dijkstra runs. Since k is a small constant, this is roughly O(n log n + (V+E) log V) vs Phase 2's O(n * (V+E) log V). At n=200 on the 400-node benchmark graph the measured speedup is **105x**.

Euclidean distance is used as a proxy for graph distance, which works well on urban road networks with roughly uniform edge density. The k=3 default is a guard against cases where a physical barrier (river, highway, road closure) makes the Euclidean nearest unit graph-expensive. All 6 city-network test locations confirmed that k=3 candidates always included the true graph-nearest unit (Test 4 below). Friedman et al. (1977) showed that k-d

tree nearest-neighbor queries run in O(log n) expected time for uniformly distributed points, with degraded performance in high dimensions. Since we are working in 2D this is not a concern.

The second optimization is dynamic edge weight support. Phase 2 stored edges as tuples, which are immutable. Phase 3 stores edges as mutable lists ([neighbor, weight]) so that update_weight(u, v, new_weight) can modify both adjacency entries in-place. Cost: O(degree(u) + degree(v)), which is O(1) for the sparse road networks we use. Dijkstra is re-run on demand each dispatch, so any weight change is automatically reflected in the next shortest-path query without any additional bookkeeping. A proper incremental shortest-path algorithm (e.g., LPA* as described by Ramalingam & Reps, 1996) would be more efficient if weights changed on every dispatch cycle, but is out of scope for this phase.

Code snippet - update_weight() and the k-d tree build:

```
def update_weight(self, u, v, new_weight):
    # scan both adjacency lists, O(degree(u) + degree(v))
    found = 0
    for edge in self._adj.get(u, []):
        if edge[0] == v:
            edge[1] = new_weight
            found += 1
    for edge in self._adj.get(v, []):
        if edge[0] == u:
            edge[1] = new_weight
            found += 1
    if found == 0:
```

```
        raise KeyError(f'no edge between {u} and {v}')


def _build(self, points, depth):   # KDTree
    if not points:
        return None
    axis = depth % 2
    points.sort(key=lambda p: p[0][axis])
    mid = len(points) // 2
    return _KDNode(
        point   = points[mid][0],
        unit_id = points[mid][1],
        left    = self._build(points[:mid],   depth + 1),
        right   = self._build(points[mid+1:], depth + 1),
    )
```

Scaling Strategy

The scaling goal was to keep dispatch latency roughly flat as fleet size grows, rather than letting it scale linearly as in Phase 2. The k-d tree approach achieves this because the number of Dijkstra runs per dispatch is fixed at k=3 regardless of fleet size. The tree rebuild after each dispatch is O(n log n), which does grow with fleet size, but its constant factor is small - sorting n=400 2D points is much cheaper than running a heap-based shortest-path on a 400-node graph.

The benchmark grid is a 20x20 grid graph with 400 nodes and approximately 760 edges. Each node has integer (x, y) coordinates equal to its grid position. Units are placed at randomly selected grid nodes. Five fleet sizes were tested: 10, 25, 50, 100, and 200 units.

Each trial runs 15 dispatch calls with a fresh fleet, repeated 3 times and averaged. The test also includes a 100-call stress test (Test 5) on a 10x10 grid with 30 units, where dispatched units are periodically returned to service to keep the fleet non-empty. No crashes or incorrect dispatches were observed in any run.

Memory usage is dominated by the k-d tree (O(n) nodes) and the Dijkstra heap (O(V+E)). Both are within acceptable bounds for the grid sizes tested. The implementation stores one tree node per available unit, so the tree shrinks as units get dispatched - there is no stale-entry accumulation. The adjacency list memory stays constant across dispatches since we only update edge weights, not the graph structure.

Testing and Validation

Six tests were run. Tests 1-5 produce verbose output; Test 6 (benchmark) runs silently and prints only the results table.

Test 1 - KDTree correctness. A 5-point set was constructed and four queries run: 1-nearest neighbor, 2-nearest neighbors, k > n (should return all points), and nearest on an empty tree (should return None, inf). All passed. The 2-NN test verified the max-heap-based k_nearest implementation by checking that results come back sorted by distance ascending.

```
samiazaman@Mac phase3 % python3 dispatch_system.py
=== Test 1: KDTree nearest-neighbor ===

  1-nearest to (2.1, 2.1): E dist=0.141  (expected E at ~0.14)
  correct: True
  1-nearest to (5.5, 2.5): D dist=0.707  (expected D at ~0.71)
  correct: True
  2-nearest to (2.1, 2.1): [('E', 0.141), ('B', 1.421)]
  first is E (closest): True
  k=10 on 5-point tree returns 5 results (expected 5)
  empty tree nearest: uid=None, dist=inf  (expected None, inf)
```

Test 2 - Dynamic edge weight update. The city road network from Phase 2 was used. Two edges were updated to simulate rush-hour congestion: HQ-Station_A raised from 5 to

50 minutes, and Station_A-Midtown from 3 to 40 minutes. The shortest path from HQ to Downtown was recomputed - it rerouted from the direct HQ->Station_A->Midtown->Downtown path to a longer but now faster alternative. Weights were then restored and the original path came back. An attempt to update a nonexistent edge (HQ-Airport) correctly raised KeyError.

```
=== Test 2: Dynamic edge weight update ===

  HQ->Downtown before: 9min  HQ -> Station_A -> Midtown -> Downtown
  HQ->Downtown after traffic: 12min  HQ -> Hospital -> Midtown -> Downtown
  route changed: True
  restored: 9min  (expected 9)
  correct: True
  nonexistent edge raises KeyError: correct
```

Test 3 - OptimizedDispatchSystem end-to-end. Same scenario as Phase 2 Test 5: three calls at mixed priorities (critical E003 at Downtown, urgent E002 at Riverside, standard E001 at Airport), five units spread across the city. The system dispatched in priority order: E003 first (POLICE_2 already at Downtown, 0.0 min), E002 second (POLICE_1 from Midtown, 5.0 min), E001 last (AMB_2 from Station_A, 6.0 min). Results matched Phase 2 exactly.

```
=== Test 3: OptimizedDispatchSystem end-to-end ===

  receiving calls:
  [RECEIVED] Call(E001, STANDARD, loc=Airport)
  [RECEIVED] Call(E002, URGENT, loc=Riverside)
  [RECEIVED] Call(E003, CRITICAL, loc=Downtown)

  queue size: 3  (expected 3)

  dispatching all:
  [DISPATCH] Call(E003, CRITICAL, loc=Downtown) -> POLICE_2 (Police Car) | travel=0.0min | route: Downtown
  [DISPATCH] Call(E002, URGENT, loc=Riverside) -> POLICE_1 (Police Car) | travel=5.0min | route: Midtown -> Downtown -> R
iverside
  [DISPATCH] Call(E001, STANDARD, loc=Airport) -> AMB_2 (Ambulance) | travel=6.0min | route: Station_A -> Airport

  --- dispatch log ---
  1. Call(E003, CRITICAL, loc=Downtown) -> POLICE_2 (0.0min) [dispatched]
  2. Call(E002, URGENT, loc=Riverside) -> POLICE_1 (5.0min) [dispatched]
  3. Call(E001, STANDARD, loc=Airport) -> AMB_2 (6.0min) [dispatched]

  first dispatched call: E003  (expected E003)
  correct: True
```

Test 4 - k-d tree accuracy vs brute-force. For all 6 city-network locations, the unit chosen by k-d tree (k=3 Euclidean candidates, then Dijkstra) was compared against the unit

chosen by brute-force Dijkstra over all 5 units. All 6 matched. This validates that k=3 is sufficient for this network - the Euclidean nearest-3 always contained the true graph-nearest unit.

```
=== Test 4: k-d tree accuracy vs brute-force ===

  call@Airport     : bf=AMB_2(6.0min)  kd=AMB_2(6.0min)  match=True
  call@Uptown      : bf=POLICE_2(7.0min)  kd=POLICE_2(7.0min)  match=True
  call@Downtown    : bf=POLICE_2(0.0min)  kd=POLICE_2(0.0min)  match=True
  call@Hospital    : bf=AMB_1(0.0min)  kd=AMB_1(0.0min)  match=True
  call@Midtown     : bf=POLICE_1(0.0min)  kd=POLICE_1(0.0min)  match=True
  call@Riverside   : bf=POLICE_2(3.0min)  kd=POLICE_2(3.0min)  match=True

  all decisions match brute-force: True
```

Test 5 - Stress test. 100 calls were dispatched on a 10x10 grid (100 nodes, ~180 edges) with 30 units. After every 10 dispatches, all dispatched units were returned to available at their current locations to prevent the fleet from running dry. All 100 calls were logged as dispatched with no crashes, no KeyErrors, and no assertion failures. The test also confirmed that the tree rebuild after each dispatch stays correct when unit locations change.

```
=== Test 5: Stress test - 100 dispatches on 10x10 grid ===

  calls logged: 100  (expected 100)
  dispatched: 37, returned to service: 7
  no crashes: True
```

Test 6 - Benchmark. Results are presented in the Performance Analysis section below.

Performance Analysis

The benchmark compared Phase 2 (DispatchSystem, n Dijkstras per dispatch) against Phase 3 (OptimizedDispatchSystem, k=3 Dijkstras + tree rebuild per dispatch) on a 20x20 grid at fleet sizes 10 to 200. Each measurement is the average ms per dispatch_next() call over 15 calls and 3 repeats.

Fleet size	Phase 2 (ms/call)	Phase 3 (ms/call)	Speedup
10	0.588 ms	0.224 ms	2.6x
25	2.762 ms	0.209 ms	13.2x
50	6.562 ms	0.160 ms	40.9x
100	14.368 ms	0.188 ms	76.4x
200	29.089 ms	0.276 ms	105.5x

Phase 2 scales linearly with fleet size - each additional unit adds one Dijkstra run. Phase 3 stays roughly flat between 0.16ms and 0.28ms across all fleet sizes because the number of Dijkstra runs is fixed at k=3. The speedup grows from 2.6x at n=10 (where the tree rebuild overhead is significant relative to only 10 Dijkstras saved) to 105x at n=200. The slight increase in Phase 3 time from n=50 to n=200 is attributable to the O(n log n) tree rebuild, whose cost grows slowly with fleet size.

Both implementations use the same Dijkstra code on the same graph, so the per-Dijkstra cost is identical. **The entire speedup comes from running 3 Dijkstras instead of n**. No shortcuts, caching, or approximations were used in the shortest-path computation itself.

Final Evaluation

The Phase 3 implementation successfully addresses all three deferred items from Phase 2. The k-d tree spatial index reduces dispatch latency from O(n * Dijkstra) to roughly O(n log n + k * Dijkstra), producing a ~100x speedup at fleet size 200. Dynamic edge weights work correctly - Test 2 confirmed rerouting under congestion and restoration after

weight rollback. The stress test validated correctness across 100 consecutive dispatches with no crashes.

Three limitations remain.

First, the same-priority FIFO issue noted in Phase 2 is still unresolved. Python's heap has no stable ordering within a priority level, so arrival order among ties is not guaranteed. The fix is a sequence-number tiebreaker in the heap tuple - straightforward but deferred.

Second, the k-d tree is rebuilt from scratch after every dispatch. For small fleets this is negligible (n=10 rebuilds are very fast), but at n=200 the rebuild is the dominant cost in Phase 3 timing. A lazy-deletion approach - marking dispatched nodes as inactive and skipping them during queries - would avoid rebuilds entirely. The tradeoff is that the tree becomes unbalanced as deleted nodes accumulate, eventually degrading query performance. A periodic full rebuild (e.g., every k dispatches) could balance both concerns.

Third, k=3 was validated against a small city network with no physical barriers. On a real road network with rivers, rail lines, or highway barriers, the Euclidean nearest unit might not be graph-reachable at all, and a higher k would be needed to ensure the true graph-nearest unit is in the candidate set. The right k depends on the network topology and should be calibrated against historical dispatch data. An alternative is to use road network distance as the spatial index metric instead of Euclidean distance, but this requires a more sophisticated spatial index structure.

On the question of time vs space trade-offs: the k-d tree adds O(n) memory per dispatch (the tree itself), which is small compared to the Dijkstra heap (O(V+E)). The choice to rebuild the tree after every dispatch (rather than lazy deletion) costs O(n log n) per

dispatch but keeps the implementation simple and avoids the correctness complexity of in-place deletion. This is the right trade-off for a v1 implementation; v2 could add lazy deletion once the baseline is proven correct at scale.

GitHub Repository

https://github.com/Samia1117/MSCS-532/tree/main/Project/phase3

References

Bentley, J. L. (1975). Multidimensional binary search trees used for associative searching. Communications of the ACM, 18(9), 509-517. https://doi.org/10.1145/361002.361007

Friedman, J. H., Bentley, J. L., & Finkel, R. A. (1977). An algorithm for finding best matches in logarithmic expected time. ACM Transactions on Mathematical Software, 3(3), 209-226. https://doi.org/10.1145/355744.355745

Ramalingam, G., & Reps, T. (1996). An incremental algorithm for a generalization of the shortest-path problem. Journal of Algorithms, 21(2), 267-305. https://doi.org/10.1006/jagm.1996.0046